

23. Synchronizace procesů pomocí semaforů a pomocí předávání zpráv, synchronizace vláken pomocí bariér a podmínkových proměnných.

Klasické synchronizační úlohy

- Producent – konzument (BoundedBuffer Problem)
 - předávání dat mezi 2 procesy (jeden data vyrábí, druhý spotřebovává)
- Čtenáři a písaři (Readers and Writers Problem)
 - Do sdílené paměti přistupují procesy, které z ní čtou nebo do ní zapisují data. Data je třeba udržet v konzistentním stavu.
- Úloha o večeřících filozofech (Dining Philosophers Problem)
 - U kulatého stolu s jídelm sedí pět (obecně n) filosofů. Každý buď přemýšlí nebo jí. Na stole je pouze pět vidliček umístěných mezi filosofy. Pro jedení potřebuje každý filosof dvě vidličky.

Semaforey se používají pro podobný účel jako mutexy, a to pro kontrolování vstupu do kritických sekcí. Ale na rozdíl od mutexu, kdy v sekci může být pouze jeden, se semaforey ze docílit, že v sekci může být více vláken.

Řešení:

- Producent - Konzument se semaforey
 - Pro výlučný přístup do skladu lze použít mutex, pro synchronizaci při prázdném a plném skladu semaforey, jejichž použití navíc odstraní aktivní čekání.
- Synchronizace pomocí fronty zpráv
 - Mechanismus mailboxů a portů
 - Mailbox je schránka pro předávání zpráv
 - Může být soukromá pro dvojici komunikujících procesů nebo sdílená více procesy
 - Jádro operačního systému vytvoří mailbox na žádost procesu a tento proces je pak jeho vlastníkem
 - Vlastník může mailbox zrušit nebo bude automaticky zrušen při skončení vlastníka
 - Port je schránka vlastněná jediným příjemcem
 - Zprávy do portu může zasílat více procesů
 - V modelu klient/server je přijímacím procesem server
 - Port zaniká ukončením přijímacího procesu
- Vzájemné vyloučení pomocí fronty zpráv
 - Mailbox se použije jako zamykací mechanismus (mutex)
 - inicializace tím, že se do fronty zpráv zašle zpráva (nebo n -zpráv)
 - procesy pak volají blokující `receive` - první kdo zprávu vybere je v KS
 - při odchodu z KS zavolá `send` a pošle novou zprávu do fronty zpráv

- Producent/Konzument s pomocí fronty zpráv
 - fronta zpráv zaručuje vzájemné vylučování při přístupu k frontě zpráv
 - implementace je naprosto triviální:
 - vytvoří se fronta zpráv `mq_init(mq_t storage, n)`, kde `n` je velikost fronty
 - producent posílá zprávy (objekty - cokoliv) do fronty zpráv a pokud je fronta plná - zablokuje ho (a neaktivně čeká)
 - probudí se jakmile fronta nebude plná
 - konzument vybírá zprávy (objekty) z fronty zpráv, dokud tam zprávy jsou - pokud už nejsou - zablokuje ho to (a neaktivně čeká)
 - probudí se jakmile fronta nebude prázdná

Synchronizace vláken pomocí bariéry

- Zárážka – synchronizace vláken na konkrétní místo v programu.
- Dokud se na bariéře nezastaví specifikovaný počet vláken, jsou všechny blokovány, následně jsou všechny najednou probuzeny

Použití:

```
// Deklarace a inicializace:
barrier_t barrier;           // deklarace bariéry
barrier_init(&barrier, 3); // stanovení počtu vláken k synchronizaci

// Použití bariér v jednotlivých vláknech:
barrier_wait(&barrier);     // čekání na daný počet vláken
// nyní poběží vlákna souběžně
```

Synchronizace vláken pomocí podmínkových proměnných

- Obdoba bariéry, ale nečeká se na určitý počet čekajících vláken, ale na splnění podmínky.
- Využívá se pomocný mutex (protože podmínková proměnná je sdílená)
- Zatímco mutexy poskytují prostředky pro synchronizaci přístupu k proměnným, podmínkové proměnné umožňují synchronizovat vlákna na základě hodnot proměnných.
 - Kdybychom neměli podmínkové proměnné, muselo by se v programu cyklicky testovat, zda už má proměnná žádanou hodnotu. Takový přístup by znamenal drastické snížení výkonu aplikace.

Příklad:

```
bool done = false; // stav dokončení
cond_t cond;       // synchronizační podmínková proměnná
mutex_t mutex;     // mutex pro zajištění vzájemné výlučnosti
                  // přístupu k proměnným
```

```
// Kód vlákna provádějícího akci, zde vypočtení hodnoty sdílené
// proměnné x:
x = calculate();           // provedení akce
mutex_lock(mutex);        // zajištění exkluzivního přístupu
done = true;              // nastavení stavu dokončení
condition_signal(cond);    // signalizace dokončení akce
mutex_unlock(mutex);      // uvolnění přístupu

// Kód vlákna čekajícího na dokončení akce:
mutex_lock(mutex);        // zajištění exkluzivního přístupu
if (!done)                // pokud akce ještě nebyla dokončena,
    condition_wait(cond); // čekej na dokončení akce
mutex_unlock(mutex);      // uvolnění přístupu
use(x);                   // použití hodnoty sdílené proměnné x
                           // vypočtené jiným vláknem
```